

CS-107 : Mini-Projet 1

Compression de données : "Quite Ok Image" Format

HAMZA REMMAL, BARBARA JOBSTMANN, JAMILA SAM

VERSION 1.7

Contents

1	Présentation	3
2	Structure et code fourni	3
2.1	Structure	3
2.2	Code fourni	3
2.2.1	Code avancé	5
2.3	Tests	6
3	Débogage de votre code	8
3.1	Hexdump	8
3.2	Diff	8
3.3	QOI Viewer	9
4	Tâche 1 : Implementation de fonctions utilitaires : ArrayUtils	10
4.1	Méthodes pour tester l'égalité	10
4.1.1	Méthodes <code>ArrayUtils::equals</code>	10
4.1.2	Autres méthodes ... ?	10
4.2	Manipulation d'entiers	10
4.2.1	Méthode <code>ArrayUtils::wrap</code>	11
4.2.2	Méthodes <code>ArrayUtils::toInt</code> et <code>ArrayUtils::fromInt</code>	11
4.3	Méthodes de concaténation	12
4.3.1	Méthodes <code>ArrayUtils::concat</code>	12
4.3.2	Méthode <code>ArrayUtils::extract</code>	12
4.3.3	Méthode <code>ArrayUtils::partition</code>	13
4.4	Méthodes de formatage	13
4.4.1	Méthode <code>ArrayUtils::imageToChannels</code>	13
4.4.2	Méthode <code>ArrayUtils::channelsToImage</code>	15
4.5	Tests	15
5	Tâche 2 : Encodeur "Quite Ok Image"	16
5.1	Structure d'un fichier "Quite Ok Image"	16

5.2	Entête du fichier	16
5.2.1	Nombre magique	17
5.2.2	Méthode <code>QOIEncoder::qoiHeader</code>	17
5.3	Algorithme de compression "Quite Ok Image"	18
5.3.1	Encodage <code>QOI_OP_RGB</code>	18
5.3.2	Encodage <code>QOI_OP_RGBA</code>	18
5.3.3	Encodage <code>QOI_OP_INDEX</code>	19
5.3.4	Encodage <code>QOI_OP_DIFF</code>	20
5.3.5	Encodage <code>QOI_OP_LUMA</code>	21
5.3.6	Encodage <code>QOI_OP_RUN</code>	22
5.4	Encodage global de l'image	23
5.5	Création du contenu du fichier	24
5.6	Tests	25
6	Tâche 3 : Décodeur "Quite Ok Image"	27
6.1	Décodage de l'entête	27
6.2	Algorithme de décompression QOI	27
6.2.1	Décodage <code>QOI_OP_RGB</code>	27
6.2.2	Décodage <code>QOI_OP_RGBA</code>	29
6.2.3	Décodage <code>QOI_OP_INDEX</code>	29
6.2.4	Décodage <code>QOI_OP_DIFF</code>	29
6.2.5	Décodage <code>QOI_OP_LUMA</code>	30
6.2.6	Décodage <code>QOI_OP_RUN</code>	30
6.3	Décodage global des données	32
6.4	Création de l'Image java	33
6.5	Tests	33
7	Extensions	34
8	Complément théorique	34
8.1	Représentation ARGB	34
8.2	Table de hachage	35
8.3	Le format "Quite Ok Image"	35
8.4	Entiers signés et entiers non signés	38

Ce document utilise des couleurs et contient des liens cliquables. Il est préférable de le visualiser en format numérique.

1 Présentation

Qu'il s'agisse de la mémoire ou de la vitesse de calcul, les ressources d'un ordinateur sont relativement limitées. En tant que futur ingénieur.e, la bonne gestion de ces ressources devrait être une de vos priorités. Au cours de ce projet, vous ferez une incursion dans le monde des fichiers. Il s'agira notamment de savoir comment un fichier contenant une image est structuré et comment un programme informatique arrive à extraire les données qu'il contient. On utilisera comme support pour ce projet deux formats d'images, le célèbre format PNG (Portable Network Graphique) et le format QOI (Quite Ok Image) créé le 24 Novembre 2021 par Dominic Szablewski. Ce dernier est un très bon exemple de compromis entre vitesse de calcul et gestion de la mémoire. Dans la prochaine section, vous trouverez une petite description de la structure du projet.

La section 8 ainsi que les transparents présentés en cours contiennent des explications dont il est utile de prendre connaissance avant d'aborder le sujet.

2 Structure et code fourni

2.1 Structure

Le projet est divisé en trois étapes :

- Fonctions utilitaires : lors de cette première étape, nous allons créer quelques fonctions dont le rôle est de nous faciliter la tâche pour les étapes suivantes.
- Encodeur QOI : dans un second temps, nous allons mettre en place la structure d'un encodeur pour le format "Quite Ok Image".
- Décodeur QOI : qu'est-ce qu'un encodeur sans son décodeur ? Nous allons donc finaliser l'outil en ajoutant le décodeur du format "Quite Ok Image".

2.2 Code fourni

Pour accéder au code fourni, suivez avec attention les instructions données dans la [description générale du projet](#).

L'archive fournie a comme structure :

```
├── references
│   ├── *.png
│   ├── *.qoi
│   └── *.txt
├── src
│   └── cs107
│       ├── ArrayUtils.java
│       ├── Diff.java
│       ├── Helper.java
│       ├── Hexdump.java
│       ├── Main.java
│       ├── QOIDecoder.java
│       ├── QOIEncoder.java
│       ├── QOISpecification.java
│       ├── SignatureChecks.java
│       └── Submit.java (fourni plus tard)
```

Vous trouverez [ici](#) la documentation javadoc du projet. Prenez connaissance du matériel fourni au travers de cette documentation (le code fourni lui-même n'est pas forcément à votre portée et il n'est pas attendu de vous que vous en compreniez les détails).

- Tout le code obligatoire devra être produit dans les fichiers `ArrayUtils.java`, `QOIDecoder.java` et `QOIEncoder.java`.
- Les entêtes des méthodes à implémenter sont fournies et **ne doivent pas être modifiées**.
- le fichier `Submit.java` servira à soumettre votre projet. Il n'est pas encore présent mais il faudra le stocker à l'endroit montré plus haut dans le projet. Lorsqu'il sera fourni, **attention à ne pas le corrompre!**.
- Le fichier fourni `SignatureChecks.java` donne l'ensemble des signatures à ne pas changer. Il sert notamment d'outil de contrôle lors des soumissions. Il permettra de faire appel à toutes les méthodes requises **sans en tester le fonctionnement**^a. Vérifiez que ce programme **compile** bien, avant de soumettre.

^aCela permet de vérifier que vos signatures sont correctes et que votre projet ne sera pas rejeté à la soumission.

2.2.1 Code avancé

Certaines fonctions nécessaires au développement du projet, telles que celles liées à la manipulation de fichiers ou d'images, sont trop avancées pour ce cours. Elles sont donc codées pour vous. À cet effet, un fichier utilitaire `Helper.java` vous est fourni contenant toutes fonctions dont l'utilité a été jugée nécessaire. Ce dernier contient :

- La définition d'un nouveau type `Image` (utilisable avec le nom `Helper.Image` depuis votre code) qui encapsulera les informations nécessaires pour encoder/décoder une image. Vous pouvez vous référer à la javadoc pour savoir comment manipuler les objets de type `Helper.Image` et avoir plus de détails sur les fonctions décrites ci-dessous.
- Une fonction pour créer une nouvelle image . Les arguments seront expliqués plus tard.

```
public static Image generateImage(int[][] data, byte channels, byte
    colorSpace)
```

- Une fonction pour extraire le contenu d'un fichier PNG. Elle prend comme argument le chemin d'accès relatif au fichier et retourne une nouvelle image.

```
public static Image readImage(String path)
```

- Une fonction pour créer une nouvelle image au format PNG. Elle prend comme argument le chemin d'accès du fichier et l'image en question. Cette fonction crée les fichiers dans le dossier `res/`

```
public static void writeImage(String path, Image image)
```

- Une méthode pour pouvoir lire le contenu d'un fichier dont le chemin d'accès est donné en argument.

```
public static byte[] read(String path)
```

- Une fonction qui crée un nouveau fichier, `Helper::write` qui prend comme paramètre le chemin d'accès du fichier et le contenu de ce dernier. De manière similaire à `Helper::writeImage`, les fichiers sont créés au niveau du dossier `res/`

```
public static void write(String path, byte[] content)
```

- Une fonction qui termine l'exécution du programme à l'aide du système d'exception de Java. Elle prend deux paramètres : Le format de message (`fmt`) et les objets nécessaires pour formater notre message. (Le formatage fonctionne de manière tout à fait similaire à `String::format`)

```
public static <T> T fail(String fmt, Object ... params)
```

L'utilisation de ces méthodes dans les contextes appropriés sera bien sûre décrite dans l'énoncé (pour le moment il s'agit juste de prendre note de leur existence).

2.3 Tests

Important : La vérification du comportement correct de votre programme vous incombe. Le fichier fourni `Main.java`, partiellement rédigé, vous servira à tester vos développements de façon simple. **Il vous revient la tâche de compléter `Main.java` en y invoquant vos méthodes de façon adéquate pour vérifier l'absence d'erreur dans votre programme.**

Lors de la correction de votre mini-projet, nous utiliserons des tests automatisés, qui passeront des données d'entrée générées aléatoirement aux différentes fonctions de votre programme. **Il y aura donc aussi des tests vérifiant comment sont gérés les *cas particuliers*.** Ainsi, il est important que votre programme traite correctement toute donnée d'entrée *valide*.

Pour ce qui est de la gestion des cas d'erreurs, il est d'usage de tester les paramètres d'entrée des fonctions; e.g., vérifier qu'un tableau n'est pas nul, et/ou est de la bonne dimension, etc. Ces tests facilitent généralement le débogage, et vous aident à raisonner quant au comportement d'une fonction. **Nous supposons que les arguments des fonctions sont valides** (sauf exceptions explicitement mentionnées).

Pour garantir cette hypothèse, nous vous invitons à utiliser les assertions Java¹. Une assertion s'écrit sous la forme :

```
assert expr;
```

avec `expr` une expression booléenne. Si l'expression est fausse, alors le programme lance une erreur et s'arrête, sinon il continue normalement. Par exemple, pour vérifier qu'un paramètre de méthode `key` n'est pas `null`, vous pouvez écrire `assert key != null;` au début du corps de la méthode (les assertions n'ont pas forcément besoin de figurer au début du corps).

Les assertions doivent être activées pour fonctionner. Ceci se fait en lançant le programme avec l'option `"-ea"`: [pour IntelliJ](#) (ou [pour Eclipse](#)) [Liens cliquables].

¹Nous aurons l'occasion d'y revenir en détail, mais leur utilisation est assez intuitive pour que nous puissions déjà y recourir

Voici un résumé des consignes/indications principales à respecter pour le codage du projet :

- Les paramètres des méthodes seront considérés comme exempts d'erreur, sauf mention explicite du contraire.
- Les entêtes des méthodes fournies doivent rester inchangées : le fichier `SignatureCheck.java` ne doit donc pas comporter de fautes de **compilation** au moment du rendu.
- En dehors des méthodes imposées, libre à vous de définir toute méthode supplémentaire qui vous semble pertinente. **Modularisez et tentez de produire un code propre !**
- La vérification du comportement correct de votre programme vous incombe. Néanmoins, nous fournissons le fichier `Main.java`, illustrant comment invoquer vos méthodes pour les tester. Les exemples de tests ainsi fournis sont non exhaustifs et vous êtes autorisé.e.s/encouragé.e.s à modifier `Main.java` pour faire d'avantage de vérifications. Vos efforts en terme de test seront aussi rétribués.
- Votre code devra respecter les conventions usuelles de nommage.
- Le projet sera codé sans le recours à des bibliothèques externes. Si vous avez des doutes sur l'utilisation de telle ou telle bibliothèque, posez-nous la question et surtout faites attention aux alternatives que IntelliJ (ou Eclipse) vous propose d'importer sur votre machine.
- Votre projet **ne doit pas être stocké sur un dépôt public** (de type github public). Pour ceux d'entre vous qui sont familiers avec git, ceci peut aussi être utile : <https://gitlab.epfl.ch/>.
- Utilisez le [forum Ed](#). Avant de poser votre question, vérifiez qu'il n'y a pas de questions similaires. Cela permet de faciliter l'utilisation du forum et de ne pas se perdre dans les questions. Respectez aussi la structure du forum: **Utilisez la catégorie Mini-projet 1**.
- Vous trouverez dans le fichier `Main.java` tous les exemples utilisés lors de la rédaction de ce document. Pensez à les exécutés un par un pour "vérifier" le comportement de votre code.

3 Débogage de votre code

Comme vous le verrez dans les prochaines sections, ce projet manipule plusieurs tableaux de `byte` (`byte[]`). Pour vous aidez à déboguer et valider votre code, 2 fichiers (`Hexdump.java` et `Diff.java`) sont mis à votre disposition. Ces deux derniers utilitaires simulent l'utilisation de 2 commandes Unix (`hexdump` et `diff`).

3.1 Hexdump

L'utilitaire `Hexdump` a pour rôle d'afficher sur le terminal l'ensemble des bytes contenu dans un tableau.

Pour pouvoir l'utiliser, 3 méthodes ont été mis à votre disposition.

La première méthode prend comme argument un tableau de `byte` (`byte[]`) et affiche le contenu de ce dernier sur le terminal. Naturellement, sa signature se résume à :

```
public static void hexdump(byte[] binary)
```

Au fur et à mesure de votre avance dans le projet, vous commencerez à manipuler des tableaux de grande taille. Ceci rend l'utilisation de la méthode ci-dessus compliquée. Pour cela, l'utilitaire `Hexdump` fournit 2 surcharges de `Hexdump::hexdump`. La première, dont la signature est :

```
public static void hexdump(byte[] binary, int start_address)
```

permet de commencer l'affichage à partir d'une certaine adresse (indice dans le tableau). La seconde, quant à elle, permet d'afficher une partie du tableau. Ceci se fait en précisant l'adresse de départ et l'adresse de fin. Sa signature est la suivante :

```
public static void hexdump(byte[] binary, int start_address, int end_address)
```

Indications : Vous utiliserez l'utilitaire `Hexdump` pour inspecter le contenu de vos fichiers ainsi que vos tableaux. Pour cela, vous devrez avoir compris la structure des fichiers ainsi que les valeurs qu'ils doivent contenir. Par exemple, lors de l'analyse d'un fichier "QOI", on s'attend à avoir le nombre magique présent au début du fichier. Si ce n'est pas le cas, inspectez le code qui s'occupe de générer cette partie de l'encodage pour y déceler les erreurs. Si vous disposez d'une référence et que vous voulez comparer votre fichier avec cette dernière, nous vous invitons à utiliser notre prochain utilitaire `Diff`.

3.2 Diff

Comme son nom le suggère, cet utilitaire permet de trouver la différence entre le contenu de deux fichiers et de l'afficher sur le terminal. L'utilisation de `Diff` peut se faire à l'aide d'une seule méthode (`Diff::diff`) dont la signature est :

```
public static void diff(String file_1, String file_2)
```

Cette méthode prend comme paramètres le chemin d'accès vers 2 fichiers. Ces derniers peuvent être de différente taille. Si c'est la cas, un **WARNING** s'affiche sur la console et l'utilitaire compare seulement la première partie des fichiers. Pour finir, si les deux fichiers ont le même contenu, la méthode `Diff::diff` affiche un **WARNING** suivi du message : "The two inputs have the same content".

3.3 QOI Viewer

Pour vous aidez à visualiser un fichier au format "Quite Ok Image", vous pouvez installer le plugin suivant du JetBrains Marketplace ([ici](#)). Pour faire, suivez les instructions ci-dessous :

1. Ouvrez IntelliJ. (Si un de vos projets s'ouvre, vous pouvez le fermer en appuyant sur *File -> Close Project* en haut de votre fenêtre)
2. À la gauche de votre fenêtre, sélectionner *plugins*.
3. Dans la section *Marketplace*, recherchez le plugin **QOI Support** et installez le.
4. Dès que l'installation est terminée, redémarrez IntelliJ. Le plugin devrait être actif.

En cliquant sur le un fichier en format `.qoi`, vous pourrez alors visualiser en clair l'image qu'il encode.

Si vous travaillez avec Eclipse, vous pouvez utiliser ce "[viewer QOI](#)" (il suffit de faire glisser vos fichiers `.qoi` dans la fenêtre du viewer).

4 Tâche 1 : Implementation de fonctions utilitaires : ArrayUtils

Comme expliqué dans les compléments (section 8.3), les images à traiter sont décomposées par canal (R,G,B et A) et linéarisées. Cela facilite et optimise l'écriture de l'encodeur (étape 2) et du décodeur (étape 3). Il vous est demandé de compléter le matériel du fichier `ArrayUtils.java` qui offrira un certain nombre de méthodes utilitaires pour réaliser ces traitements. Ce fichier contient déjà les signatures, il vous suffira donc de compléter le corps des méthodes. Vous pouvez bien sûr en ajouter à votre guise mais **il ne faut en supprimer aucune**.

Pour tout le projet, vous manipulerez les bytes au moyen d'opérateurs binaires (voir à ce sujet les transparents du cours et la section 8.1).

4.1 Méthodes pour tester l'égalité

Le but de cette section est de doter notre "boîte à outils" des méthodes qui vont nous permettre de tester l'égalité entre le contenu de certains tableaux.

4.1.1 Méthodes `ArrayUtils::equals`

Comme leur nom l'indique, le but de ces deux méthodes est de tester l'égalité entre le contenu de deux tableaux. Elles ont comme signatures :

```
public static boolean equals(byte[] a1, byte[] a2)
```

```
public static boolean equals(byte[][] a1, byte[][] a2)
```

Ces deux méthodes fonctionnent de manière similaire. Si le contenu des deux tableaux est identique, la valeur `true` est retournée, autrement ce sera la valeur `false`.

Comment gérer le cas de `null` ? Si les deux paramètres de votre méthode contiennent la référence spéciale `null`, la valeur de retour doit être `true`. Nous adoptons la sémantique que puisque les deux références sont identiques, les deux tableaux sont égaux même s'ils n'ont pas de contenu. En revanche, si un seul des paramètres vaut `null`, le programme devra s'arrêter en utilisant le mécanisme d'assertions précédemment décrit.

Votre implémentation de `ArrayUtils::equals` devra se faire en utilisant les structures de base de Java. Vu que le but est d'apprendre à coder ces outils par vous-même, il est ici **interdit d'utiliser des méthodes prédéfinies offertes par l'API de Java**.

4.1.2 Autres méthodes ... ?

Les méthodes proposées ci-dessus ne permettent d'effectuer le test d'égalité qu'entre deux tableaux de `byte` ou de `byte[]`. Si plus tard vous le jugez nécessaire, vous pouvez surcharger la définition de `ArrayUtils::equals` en gardant le même nom pour la méthode, mais en changeant le type dans la signature de la méthode.

4.2 Manipulation d'entiers

Le but des trois prochaines méthodes est de permettre de convertir une partie des types entiers en tableau et vice versa.

4.2.1 Méthode `ArrayUtils::wrap`

La première méthode consiste à «envelopper» un `byte` dans un tableau. Le comportement attendu est illustré par l'exemple fourni.

```
// ArrayUtils::wrap's signature
public static byte[] wrap(byte value)
// Example
byte a = 1;
byte[] wrapped_a = ArrayUtils.wrap(a); // wrapped_a = [1]
```

4.2.2 Méthodes `ArrayUtils::toInt` et `ArrayUtils::fromInt`

Il sera utile pour la suite de pouvoir coder les données soit sous la forme d'un tableau de `byte` soit sous la forme plus concise d'un entier (`int`). Comme vu dans le cours, le type `int` est 4 fois plus grand que le type `byte`. Pour passer de l'un à l'autre, nous devrons adopter une convention sur l'ordre des bytes formant l'entier. Ces conventions sont connues sous le nom de «boutisme» (ou «endianness» en anglais). Pour ce projet, le format pour manipuler les entiers sera **big-endian** (byte le plus significatif d'abord, voir les exemples donnés ci-dessous).

Il vous est demandé de coder les deux méthodes ci-dessous pour la manipulation des entiers. La première (`ArrayUtils::toInt`) prend comme paramètre un tableau de `byte` et reconstitue l'entier correspondant. La seconde (`ArrayUtils::fromInt`) prend comme paramètre un entier et le décompose en `byte`.

Vous coderez ces méthodes au moyen d'**opérateurs binaires**. À ce sujet, référez-vous aux transparents du cours ainsi qu'aux compléments en 8.1 et 8.4. Les exemples fournis illustrent les comportements attendus.

`ArrayUtils::toInt` :

```
// ArrayUtils::toInt's signature
public static int toInt(byte[] bytes)
// Example
byte[] array = {(byte) 0x7B, (byte) 8, (byte) 4, (byte) 7};
// pareil que : byte[] array = {123, 8, 4, 7};
int value = ArrayUtils.toInt(array); // value = 0x7B_08_04_07 / 2064122887
```

`ArrayUtils::fromInt` :

```
// ArrayUtils::fromInt's signature
public static byte[] fromInt(int value)
// Example
int value = 0xBC614E; // 12345678
byte[] array = ArrayUtils.fromInt(value); // array = [0x00, 0xBC, 0x61, 0x4E]
// array = [0, -68, 97, 78]
```

Les hypothèses présumées correctes et qui sont donc à vérifier au moyen d'assertions pour la méthode `ArrayUtils::toInt` sont : que le tableau `bytes` ne vaut pas `null` et que sa taille vaut 4.

4.3 Méthodes de concaténation

Maintenant que nous pouvons tester l'égalité des tableaux et manipuler des entiers, nous pouvons implémenter les méthodes utiles à la manipulation de tableaux. Une de ces opérations est la **concaténation**.

4.3.1 Méthodes `ArrayUtils::concat`

Il vous est d'abord demandé de coder, en utilisant le principe de la *surcharge*, deux versions d'une méthode `concat` dont l'une prend en argument un nombre indéterminé de tableaux de `byte` et l'autre un nombre indéterminé de `byte`. Les comportements attendus sont illustrés par les exemples fournis.

```
// ArrayUtils::concat's signature
public static byte[] concat(byte[] ... tabs)
// Example
byte[] tab1 = new byte[]{1, 2, 3};
byte[] tab2 = new byte[0];
byte[] tab3 = new byte[]{4};
// tab = [1, 2, 3, 4]
byte[] tab = ArrayUtils.concat(tab1, tab2, tab3);
```

```
// ArrayUtils::concat's signature
public static byte[] concat(byte ... bytes)
// Example
byte[] tab = ArrayUtils.concat((byte)1, (byte)2, (byte)3, (byte)4); // tab =
    [1, 2, 3, 4]
```

Ces méthodes `ArrayUtils::concat` utilisent la notion d'*ellipse*, ce qui nous permet de concaténer autant de données que l'on souhaite et en rend l'utilisation plus aisée.

Les hypothèses présumées correctes (à vérifier par des assertions) pour `ArrayUtils::concat` sont qu'aucun des objets `tabs`, `tabs[i]` et `bytes` ne vaut `null`.

4.3.2 Méthode `ArrayUtils::extract`

Dans un second temps, nous nous intéressons à pouvoir extraire un tableau d'un autre. Pour cela, il vous est demandé d'implémenter la méthode `ArrayUtils::extract` qui prend comme paramètre un tableau, la position à laquelle l'extraction débutera et la taille du tableau à extraire. Le comportement général attendu est illustré par l'exemple fourni.

```
// ArrayUtils::extract's signature
public static byte[] extract(byte[] input, int start, int length)
// Example
byte[] tab = new byte[]{1, 2, 3, 4, 5, 6, 7, 8};
// extracted = [3, 4, 5, 6, 7]
byte[] extracted = ArrayUtils.extract(tab, 2, 5);
```

Les hypothèses présumées correctes (à vérifier par des assertions) pour `ArrayUtils::extract` sont :

- que `input` ne vaut pas `null`;
- que `start` est une valeur licite d'indice pour `input` ;
- que `length` est une valeur positive ou nulle ;
- et que `start+length` vaut au plus `input.length`.

4.3.3 Méthode `ArrayUtils::partition`

Le projet nécessitera de pouvoir partitionner des tableaux en sous-tableaux de diverses tailles. Il vous est demandé pour cela de coder la méthode `ArrayUtils::partition` qui prend comme paramètre le tableau à partitionner et les tailles respectives de chaque partition dans le tableau final. L'exemple fourni illustre le comportement général attendu.

```
// ArrayUtils::partition's signature
public static byte[][] partition(byte[] input, int ... sizes)
// Example
byte[] tab = new byte[]{1, 2, 3, 4, 5, 6, 7, 8, 9};
// partitions = [[1, 2, 3], [4], [5, 6], [7], [8, 9]]
byte[][] partitions = ArrayUtils.partition(tab, 3, 1, 2, 1, 2);
```

Les hypothèses présumées correctes (à vérifier par des assertions) pour `ArrayUtils::partition` sont :

- que `input` et `sizes` ne valent pas `null` ;
- que la somme des entiers de `sizes` vaut la longueur de `input`.

4.4 Méthodes de formatage

Dans la prochaine étape du projet (`QOIEncoder`), vous aurez à traiter les pixels d'une image un à un. Pour vous faciliter les tâches à venir, vous aurez à implémenter deux méthodes qui permettront de changer la structure d'un tableau, tout en préservant l'information qu'il contient.

La première méthode aura pour rôle de séparer les différents canaux de couleur d'une image (pour pouvoir traiter séparément les composantes R,G,B et A de ses pixels). La seconde permettra de faire l'opération inverse: à savoir restituer le format usuel d'une image sous la forme d'un tableau à deux dimensions de pixels, en partant de la représentation produite par la première méthode.

4.4.1 Méthode `ArrayUtils::imageToChannels`

Soit une image codifiée comme un tableau à deux dimensions `int[][] input`. Chacun des entiers codifie un *pixel* qui est constitué de **4 bytes**. Ces derniers représentent la valeur d'intensité de chaque canal R,G,B et A d'un pixel donné. La méthode `ArrayUtils::imageToChannels` qu'il vous est demandé de coder ici a pour but de créer un nouveau tableau à deux dimensions, de type `byte[][]` de sorte à ce que chaque colonne contienne l'ensemble des bytes d'un canal donné (par exemple, l'ensemble des bytes du canal R de tous les pixels du tableau `input`). Chaque colonne représente quant à elle un canal.

Soit l'image en entrée :

```
input = { {p[0][0], ....., p[0][w-1]},
          {p[1][0], ....., p[1][w-1]},
          ....
          {p[h-1][0], ....., p[h-1][w-1]}
        }
// p[x][y] is the int value of a pixel
```

Le tableau créé sera :

```
{ {r[0][0], g[0][0], b[0][0], a[0][0]},
  {r[0][1], g[0][1], b[0][1], a[0][1]},
  ...
  {r[0][w-1], g[0][w-1], b[0][w-1], a[0][w-1]}
  {r[1][0], g[1][0], b[1][0], a[1][0]}
  ....
  {r[1][w-1], g[1][w-1], b[1][w-1], a[1][w-1]}
  ...
  {r[h-1][0], g[h-1][0], b[h-1][0], a[h-1][0]}
  ....
  {r[h-1][w-1], g[h-1][w-1], b[h-1][w-1], a[h-1][w-1]}
};
//r[x][y] is the value of p[x][y]'s R component
//g[x][y] is the value of p[x][y]'s G component
//b[x][y] is the value of p[x][y]'s B component
//a[x][y] is the value of p[x][y]'s A component
```

Les données du tableau d'entrée sont donc linéarisées (on a "brisé" la structure à deux dimensions pour les données des pixels eux-mêmes). Tout ceci se résume à coder une méthode dont la signature est :

```
// ArrayUtils::imageToChannels signature
public static byte[][] imageToChannels(int[][] input)
// Example
int[][] input = {{1, 2, 3, 4, 5},
                 {6, 7, 8, 9, 10},
                 {11, 12, 13, 14, 15}};
// output = [[0, 0, 1, 0], [0, 0, 2, 0], [0, 0, 3, 0],
//           [0, 0, 4, 0], [0, 0, 5, 0], [0, 0, 6, 0],
//           [0, 0, 7, 0], [0, 0, 8, 0], [0, 0, 9, 0],
//           [0, 0, 10, 0], [0, 0, 11, 0], [0, 0, 12, 0],
//           [0, 0, 13, 0], [0, 0, 14, 0], [0, 0, 15, 0]]
byte[][] output = ArrayUtils.imageToChannels(input);
```

Vous noterez que le fichier fourni `QOISpecification.java` fournit les constantes utiles `r,g,b` et `a`. Il faudra respecter l'ordre de stockage des colonnes tels que suggéré par les exemples fournis.

Les hypothèses à vérifier comme correctes pour `ArrayUtils::imageToChannels` sont :

- que `input` et `input[i]` ne valent pas `null` ;
- que toutes les lignes de `input` sont de même taille.

4.4.2 Méthode `ArrayUtils::channelsToImage`

De manière complémentaire, une méthode devra être implémentée qui a pour rôle de rétablir la structure en 2 dimensions d'une image dont les canaux R,G,B et A ont été décomposés. Elle prendra comme paramètre un tableau `byte[][]` (ayant la même structure que le tableau retourné par `ArrayUtils::image_to_channels`), mais également la longueur et la largeur du tableau à créer.

Contrairement à d'autres opérations, la méthode `ArrayUtils::channelsToImage` ne peut pas se limiter au tableau de `bytes` pour reconstruire la structure en 2 dimensions de l'image. Ces dimensions ne peuvent être inférées sur la base de cette seule information. Les paramètres additionnels permettent donc de spécifier les dimensions du tableau à construire.

```
// ArrayUtils::channelsToImage's signature
public static int[][] channelsToImage(byte[][] input, int height, int width)
// Example
byte[][] formatted_input = {
    {0, 0, 1, 0}, {0, 0, 2, 0}, {0, 0, 3, 0},
    {0, 0, 4, 0}, {0, 0, 5, 0}, {0, 0, 6, 0},
    {0, 0, 7, 0}, {0, 0, 8, 0}, {0, 0, 9, 0},
    {0, 0, 10, 0}, {0, 0, 11, 0}, {0, 0, 12, 0},
    {0, 0, 13, 0}, {0, 0, 14, 0}, {0, 0, 15, 0}};
// output = [[1, 2, 3, 4, 5], [6, 7, 8, 9, 10], [11, 12, 13, 14, 15]]
int[][] output = ArrayUtils.channelsToImage(formatted_input, 3, 5);
```

Les hypothèses présumées correctes (à vérifier par des assertions) pour `ArrayUtils::channelsToImage` sont :

- que `input` et `input[i]` ne valent pas `null` ;
- que la taille de `input[i]` vaut 4 ;
- et que la taille de `input` vaut `height * width`.

4.5 Tests

Pour tester le comportement de vos utilitaires, utilisez le programme `Main.java`. Vous pouvez y invoquer toute méthodes utile aux tests. Certains exemples de tests sont fournis au travers de méthodes que vous pouvez utiliser et dont vous pouvez vous inspirer. Ces exemples ne sont pas exhaustifs. **complétez-les** selon ce qu'il vous semble judicieux pour tester l'ensemble des méthodes implémentées dans cette première partie du projet.

5 Tâche 2 : Encodeur “Quite Ok Image”

Maintenant que nous sommes bien outillés avec les utilitaires de `ArrayUtils`, nous pouvons entrer dans le vif du sujet et commencer l’implémentation de l’*encodeur*.

À partir d’une image (de type `Helper.Image`), le but est d’en produire une version compressée, selon le format “Quite Ok Image”(“QOI”). Conformément à ce format, le résultat sera un tableau de `bytes` constitué :

- d’un ensemble de `bytes` représentant une *entête* (“header” en anglais) ;
- d’un ensemble de `bytes` codifiant les données compressées ;
- et d’un ensemble de `bytes` représentant une *signature* et qui marquent la fin d’un fichier “Quite Ok Image”.

Tout d’abord, nous allons commencer par créer l’entête. Nous nous préoccupons ensuite de l’encodage des données qui constituent l’image. Nous terminerons en ajoutant la *signature* et en combinant le tout.

Il est très important de tester votre code méthode par méthode au fur et à mesure que vous progressez. Référez-vous aux sections 3 et 5.6 pour des indications essentielles à ce sujet. **Prenez connaissance en particulier des différents petits fichiers de données mis à votre disposition pour faciliter le débogage** (tels que décrits dans 5.6).

5.1 Structure d’un fichier “Quite Ok Image”

Voici tout d’abord quelques précisions sur les trois parties composant un fichier au format **Quite Ok Image**. La première partie, l’*entête* du fichier, contient ce que l’on appelle des **méta-données** (“metadata” en anglais). Il s’agit d’informations additionnelles sur les données contenues dans le fichier. La seconde partie comprend les données relatives à l’image, c’est à dire **la couleur de chaque pixel**. Finalement, le fichier se termine à l’aide d’une suite de bytes qui correspondent à un marqueur de fin de fichier (**EOF**, “End-Of-File”).

Entête (header)	
Contenu de l’image	
EOF (End-Of-File)	

5.2 Entête du fichier

La plupart des formats de fichiers ou de protocoles utilise un *entête* (“header”). Cette partie contient des informations additionnelles telles que la dimension de l’image ou bien l’algorithme de compression utilisé.

Pour le format **Quite Ok Image**, l’entête est constitué de 5 composantes.

Nombre magique	4 bytes
Largeur de l’image	4 bytes
Hauteur de l’image	4 bytes
Nombre de canaux	1 byte
Espace de couleur	1 byte

En imagerie numérique, un espace de couleur est une notion abstraite qui permet la construction d’une palette de couleur. Ce dernier est formé d’une base à 3 composantes et chaque couleur est ainsi associée à une coordonnée dans la dite base. Comprendre cette notion n’est pas indispensable pour le bon déroulement du projet. Pour les plus curieux, cet [article](#) décrit le concept avec plus de détails.

5.2.1 Nombre magique

Le *nombre magique* ("magic number" en anglais) est une suite de **bytes** qui permet d'identifier un protocole ou un format de fichiers. De nombreux formats qui vous sont familiers utilisent un tel nombre. Voici quelques exemples :

Formats ou Protocoles	Nombre Magique
JVM (*.class)	0xCAFEBABE
PNG	0x89504E470D0A1A0A
ZIP	'P' 'K'

Pour **Quite Ok Image**, le nombre magique est constitué de 4 bytes.

Formats ou Protocoles	Nombre Magique
Quite Ok Image	'q' 'o' 'i' 'f'

5.2.2 Méthode `QOIEncoder::qoiHeader`

Pour pouvoir créer la structure de l'entête du fichier, il vous est demandé de coder la méthode `QOIEncoder::qoiHeader`. Cette dernière doit prendre une image et extraire les informations nécessaires pour former l'entête pour la version **QOI** de l'image.

```
// QOIEncoder::qoiHeader's signature
public static byte[] qoiHeader(Image image)
// Example
Image image = Helper.generate_image(new int[32][64], QOISpecification.RGB,
    QOISpecification.sRGB);
// header = [113, 111, 105, 102, 0, 0, 0, 64, 0, 0, 0, 32, 3, 0]
byte[] header = QOIEncoder.qoiHeader(image);
```

Les hypothèses à vérifier pour `QOIEncoder::qoiHeader` sont :

- que **image** n'est pas **null** ;
- que le nombre de canaux encodant l'image ne diffère pas des valeurs des constantes `QOISpecification.RGB` et `QOISpecification.RGBA` ;
- que la valeur encodant l'espace de couleur ne diffère pas des valeurs `QOISpecification.sRGB` et `QOISpecification.ALL`.

La documentation du type `Helper.Image` est à votre disposition (voir la [javadoc](#)). Les fonctions associées aux données de ce type s'utilisent avec la notation pointée (la même chose que ce que vous faites pour les fonctionnalités du type `String`). Par exemple, si **image** est de type `Helper.Image` :

- `image.data()` : retourne le tableau de pixels associé à l'image (un `int[][]`);
- `image.channels()` : retourne le nombre de canaux associés à l'image;
- `image.color_space()` : retourne l'entier correspondant à l'espace de couleur.

Indications : vous veillerez à vous aider des méthodes utilitaires déjà codées. Vous avez à disposition dans le fichier `QOISpecification.java` les constantes `RGBA` et `RGB` pour les valeurs du nombre de canaux ainsi que les constantes `sRGB` et `ALL` pour les valeurs correspondantes aux espaces de couleurs.

5.3 Algorithme de compression "Quite Ok Image"

Nous pouvons maintenant passer à l'algorithme de compression **Quite Ok Image**. Ce dernier s'appuie sur la ressemblance entre pixels. Pour pouvoir bénéficier d'un taux de compression élevé, le concepteur du format (Dominic Szablewski) utilise **six** types d'encodages différents, chacun d'eux apporte un bénéfice à l'algorithme global. Dans les prochaines sections, nous allons donner une brève description de chacun de ces 6 types et comment nous les manipulerons dans ce projet.

5.3.1 Encodage *QOI_OP_RGB*

Cet encodage est le plus simple. Il a comme structure :

QOI_OP_RGB											
byte[0]								byte[1]	byte[2]	byte[3]	
7	6	5	4	3	2	1	0	7 .. 0	7 .. 0	7 .. 0	
QOI_OP_RGB TAG								RED	GREEN	BLUE	

Pour encoder un pixel au format *QOI_OP_RGB*, vous aurez à mettre en oeuvre la méthode: `QOIEncoder::qoiOpRGB`, dont la signature est la suivante :

```
// QOIEncoder::qoiOpRGB's signature
public static byte[] qoiOpRGB(byte[] pixel)
// Example
byte[] pixel = {100, 0, 55, 0}; // order is R,G,B,A
// encoding = [-2, 100, 0, 55]
byte[] encoding = QOIEncoder.qoiOpRGB(pixel);
```

Cette dernière prend un pixel comme parametre et retourne l'encodage *QOI_OP_RGB* correspondant.

L'hypothèse présumée correcte pour `QOIEncoder::qoiOpRGB` est que `pixel` est de taille 4.

Pensez à faire bon usage des utilitaires codés. La constante `QOI_OP_RGB_TAG` qui correspond au "tag" spécifique à ce type de blocs (0b11_11_11_10), est fourni dans le fichier `QOISpecification.java`.

5.3.2 Encodage *QOI_OP_RGBA*

De manière tout à fait similaire à l'encodage *QOI_OP_RGB*, l'encodage *QOI_OP_RGBA* contient en plus le canal alpha. La structure en mémoire est la suivante :

QOI_OP_RGBA												
byte[0]								byte[1]	byte[2]	byte[3]	byte[4]	
7	6	5	4	3	2	1	0	7 .. 0	7 .. 0	7 ..0	7 .. 0	
QOI_OP_RGBA TAG								RED	GREEN	BLUE	ALPHA	

Similairement à `QOIEncoder::qoiOpRGB` et en ajoutant les modifications nécessaires, vous aurez à implémenter la méthode `QOIEncoder::qoiOpRGBA` dont la signature est :

```
// QOIEncoder::qoiOpRGBA's signature
public static byte[] qoiOpRGBA(byte[] pixel)
// Example
byte[] pixel = {100, 0, 55, 73}; // order R,G,B,A
// encoding = [-1, 100, 0, 55, 73]
byte[] encoding = QOIEncoder.qoiOpRGBA(pixel);
```

La constante `QOI_OP_RGBA` qui correspond au "tag" spécifique à ce type de blocs vaut `0b11_11_11_11`.

L'hypothèse présumée correcte pour `QOIEncoder::qoiOpRGBA` est que `pixel` est de taille 4.

5.3.3 Encodage *QOI_OP_INDEX*

Comme vous le verrez dans la prochaine section, l'algorithme de compression utilise une **table de hachage** (voir aussi les transparents du cours ainsi que les sections 8.2 et 8.3 dans les compléments). Pour pouvoir gagner de la place en mémoire, l'encodage *QOI_OP_INDEX* permet d'encoder un indice utilisé dans la table de hachage sous la forme suivante :

QOI_OP_INDEX									
byte[0]									
7	6			5	4	3	2	1	0
QOI_OP_INDEX_TAG				INDEX					

Par construction (selon l'algorithme), l'indice encodé ne peut dépasser la valeur 64 (non comprise). La constante `QOI_OP_INDEX` qui correspond au "tag" spécifique à ce type de blocs vaut `0b00_00_00_00`.

Concrètement, vous coderez la méthode `QOIEncoder::qoiOpIndex` dont la signature est :

```
// QOIEncoder::qoiOpIndex's signature
public static byte[] qoiOpIndex(byte index)
// Example
byte index = 43;
// encoding = [43]
byte[] encoding = QOIEncoder.qoiOpIndex(index);
```

Cette méthode prend en paramètre un indice (clé) dans une table de hachage (il s'agit de `INDEX` dans le schéma ci-dessus) et retourne un `byte` (enveloppé dans un tableau) et calculé comme expliqué par le schéma et l'exemple.

L'hypothèse présumée dans `QOIEncoder::qoiOpIndex` est que l'indice est valide pour l'algorithme (non négatif et valant au plus 63).

5.3.4 Encodage *QOI_OP_DIFF*

Dans la plupart des images courantes, les pixels adjacents se ressemblent très souvent. Un pixel peut même ne différer de celui qui le précède que de quelques petites unités de couleurs (un peu plus rouge ou un peu moins bleu par exemple). Pour ces cas précis, le format "QOI" utilise l'encodage *QOI_OP_DIFF*. Ce dernier encode la variation des valeurs de chaque canal entre deux pixels consécutifs. Au moment du décodage, il nous suffira donc de connaître la valeur du pixel précédent et des variations pour pouvoir retrouver le pixel suivant.

QOI_OP_DIFF										
byte[0]										
7	6				5	4	3	2	1	0
QOI_OP_DIFF_TAG					dr		dg		db	

La constante *QOI_OP_DIFF* qui correspond au "tag" spécifique à ce type de blocs vaut `0b01_00_00_00`. Pour former cet encodage, il suffit de passer comme paramètres à la méthode `QOIEncoder::qoiOpDiff`, les variations de chaque canal.

La notation *dr* désigne la variation du canal "rouge", *dg* celle du canal "vert" et *db* celle du canal "bleu". Les variations ci-dessus sont calculées en utilisant la formule suivante :

$$d\# = \text{current_pixel}[\#] - \text{previous_pixel}[\#]$$

$$d\# = d\# + 2 \text{ (voir plus bas l'explication)}$$

La signature de la méthode à coder est :

```
/* QOIEncoder::qoiOpDiff's signature, diff = {dr',dg',db'} */
public static byte[] qoiOpDiff(byte[] diff)
// Example
byte[] diff = {-2, -1, 0};
// encoding = [70]
byte[] encoding = QOIEncoder.qoiOpDiff(diff);
```

Pour pouvoir être encodées à l'aide de `QOIEncoder::qoiOpDiff`, les variations dans les canaux devront respecter certaines contraintes.

Par construction, les variations de chaque canal ne peuvent dépasser une certaine limite. Cette limite découle principalement de la taille que prend $d\#'$ dans l'encodage final. Pour **QOI_OP_DIFF**, chaque variation est stockée sur 2 bits; ce qui finalement permet de stocker jusqu'à 4 valeurs différentes. Le format défini donc comme contrainte :

$$-3 < d\# < 2 \Rightarrow d\# \in \{-2, -1, 0, 1\}$$

La méthode `qoiOpDiff` assurera que les valeurs des variations passées en paramètres soient stockées avec un décalage de 2 dans le bloc construit (c'est à dire qu'elle leur ajoute 2 à chacune). L'ajout de ces décalages est une simple astuce simplificatrice (il s'agit de garantir que les nombres manipulés restent positifs, ce qui en simplifie la manipulation). Les opérations sur les valeurs des différences se font en **arithmétique modulaire** (voir la section 8.4). Il faut donc convertir le résultat des calculs (ajout du décalage) en `byte`.

Les hypothèses présumées correctes (à vérifier par des assertions) pour `QOIEncoder::qoiOpDiff` sont :

- que `diff` ne vaut pas `null` ;
- que `diff` contient 3 bytes (la variation pour chacun des canaux R,G et B) ;
- que le contenu de `diff` respecte les contraintes prévues pour le format `QOIEncoder::qoiOpDiff`; c-à-d que les variations de chaque canal doivent respecter la limite définies ci-dessus.

5.3.5 Encodage *QOI_OP_LUMA*

Comme dans la section précédente, l'encodage *QOI_OP_LUMA* est basé sur la similarité de deux pixels adjacents. La seule différence entre les deux encodages apparaît au niveau des contraintes appliquées aux variations.

QOI_OP_LUMA																				
byte[0]								byte[1]												
7	6						5	4	3	2	1	0	7	6	5	4	3	2	1	0
QOI_OP_LUMA_TAG							dg				dr - dg				db - dg					

La constante `QOI_OP_LUMA` qui correspond au "tag" spécifique à ce type de blocs vaut `0b10_00_00_00`. De manière similaire à `QOIEncoder::qoiOpDiff`, la méthode devra prendre comme paramètre la variation dans chacun des canaux et a comme signature :

```
// QOIEncoder::qoiOpLuma's signature: diff={dr', dg', db'}
public static byte[] qoiOpLuma(byte[] diff)
// Example
byte[] diff = {19, 27, 20};
// encoding = [-69, 1]
byte[] encoding = QOIEncoder.qoiOpLuma(diff);
```

Pour pouvoir être encodées à l'aide de `QOIEncoder::qoiOpLuma`, les variations dans les canaux devront respecter certaines contraintes.

Ces dernières s'imposent en raison de la taille de chaque variation dans l'encodage final.

$$\begin{aligned} -33 < dg' < 32 \\ -9 < dr' - dg' < 8 \\ -9 < db' - dg' < 8 \end{aligned}$$

Pour finir, la valeur de dg' doit être stockée avec un décalage de 32. Les valeurs de $dr' - dg'$ et $db' - dg'$, quant à elles, doivent être stockées avec un décalage de 8.

Les hypothèses présumées correctes (à vérifier par des assertions) pour `QOIEncoder::qoiOpLuma` sont :

- que `diff` ne vaut pas `null`;
- que `diff` contient 3 bytes ;
- et que le contenu de `diff` respecte les contraintes du format `QOIEncoder::qoiOpLuma`; c-à-d les conditions mentionnées ci-dessus.

5.3.6 Encodage *QOI_OP_RUN*

Pour finir, l'encodage *QOI_OP_RUN* encode le nombre de répétitions successives d'un pixel dans une image. Ce dernier type de blocs est lui aussi soumis à certaines contraintes.

QOI_OP_RUN									
byte[0]									
7	6			5	4	3	2	1	0
QOI_OP_RUN_TAG				count					

Le “tag” *QOI_OP_RUN* vaut `0b11_00_00_00`.

Pour pouvoir optimiser l'encodeur, le nombre de répétitions ne doit pas être égal à zéro (répéter 0 fois un pixel n'apporte en effet aucun bénéfice). De manière similaire à *QOI_OP_DIFF* et *QOI_OP_LUMA*, le nombre de répétition doit être stocké avec un décalage de -1 effectué par la méthode `qoiOpRun`. Un problème peut alors néanmoins apparaître. En effet, si la valeur de *count* est égale à 64 ou 63 (avant application du décalage), l'encodage de *QOI_OP_RUN* correspond alors à la valeur de certains “tags” (concrètement à *QOI_OP_RGB_TAG* et *QOI_OP_RGBA_TAG*). Cela doit être évité pour éviter des ambiguïtés qui compliquent le décodage.

Pour cela, ces valeurs sont considérées comme interdites et la valeur de *count* vaudra au plus 62.

Ainsi, si par exemple 113 pixels consécutifs sont égaux, l'algorithme d'encodage utilisera deux blocs *QOI_OP_RUN*, l'un avec un *count* valant 62 et l'autre avec un *count* valant 50 (le premier pixel de la séquence sera quant à lui encodé avec un autre bloc que *QOI_OP_RUN*).

La méthode `QOIEncoder::qoiOpRun` prendra comme paramètre le nombre de répétitions d'un pixel et aura comme signature :

```
// QOIEncoder::qoiOpRun's signature
public static byte[] qoiOpRun(byte count)
// Example
byte count = 41;
// encoding = [-24]
byte[] encoding = QOIEncoder.qoiOpRun(count);
```

L'hypothèse présumée correcte pour `QOIEncoder::qoiOpRun` est que *count* est comprise entre 1 et 62 (compris).

5.4 Encodage global de l'image

Maintenant que nous disposons des 6 méthodes de compression utile, il convient de les faire coopérer de façon adéquate pour pouvoir exploiter leurs potentiels au maximum. Pour cela, nous vous proposons une description simple et basique de l'algorithme. Libre à vous de l'implémenter comme il vous semble le mieux de le faire.

L'algorithme "Quite Ok Image":

Dans cet algorithme, vous aurez à traiter vos images de manière linéaire, c-à-d que les pixels seront traités consecutivement et que chaque pixel ne sera traité qu'une seule fois.

Étape 1 (Initialisation) :

Avant de commencer à traiter votre image, vous aurez à définir l'environnement d'exécution de votre algorithme. Ce dernier consiste à mettre en place un ensemble de *variables locales* nécessaires.

La première d'entre elles sert à mémoriser la valeur du *pixel précédent* (`byte[]`), elle devra être initialisée à l'aide du pixel défini dans `QOISpecification.START_PIXEL`.

Vous définirez aussi une *table de hachage* (`byte[64][4]`). Elle devra être vide au début et est destinée à stocker les pixels traités à l'aide de la fonction de hachage prédéfinie (`QOISpecification::hash`). Cette table sera utilisée pour former les encodages de types `QOI_OP_INDEX`.

Finalement vous aurez à définir un *compteur* (`int`), dont l'utilité principale est de former les blocs d'encodages `QOI_OP_RUN`. D'autres variables peuvent s'ajouter à ces dernières selon ce qui vous semble judicieux.

Étape 2 (Traitement des pixels) :

Le traitement des pixels peut ensuite débiter. Il s'agit de parcourir les pixels un par un et, chemin faisant, essayer de construire les différents blocs possibles dans l'ordre de priorité descendant suivant des types de blocs :

- nombre de pixels similaires consécutifs (`QOI_OP_RUN`);
- indice du pixel courant dans la table de hachage (`QOI_OP_INDEX`);
- différence entre 2 pixels (`QOI_OP_DIFF`)
- différence entre 2 pixels (`QOI_OP_LUMA`)
- pixel complet (`QOI_OP_RGB` ou `QOI_OP_RGBA`)

Plus précisément, pour chaque pixel :

1. **Si** `pixel == pixel précédent`, augmenter le compteur et tester si un `QOI_OP_RUN` peut être construit (compteur atteignant la limite de 62 ou dernier pixel atteint), puis passer au pixel suivant, **sinon** tester si un `QOI_OP_RUN` peut être construit et aller à l'étape 2.
2. **Si** `pixel` est dans la table de hachage (il a la même valeur que le pixel stocké à la position qui lui revient selon sa clé de hachage) , créer un bloc `QOI_OP_INDEX` et passer au pixel suivant, **sinon** ajouter le pixel à la table de hachage et aller à l'étape 3.
3. **Si** le canal alpha est le même entre le pixel courant et le précédent, et que la différence entre ces pixels est faible (selon les critères spécifiques aux blocs `QOI_OP_DIFFa`) alors créer un bloc `QOI_OP_DIFF` et passer au pixel suivant, **sinon** aller à l'étape 4.

4. Si le canal alpha est le même entre le pixel courant et le précédent, et que la différence entre ces pixels correspond aux critères des blocs *QOI_OP_LUMA*, alors créer un bloc de ce type et passer au pixel suivant, **sinon** aller à l'étape 5.
5. Si le canal alpha est le même entre le pixel courant et le précédent, créer un bloc *QOI_OP_RGB* (la valeur du canal alpha étant celle commune aux deux pixels) et passer au pixel suivant, **sinon** aller à l'étape 6.
6. créer un bloc *QOI_OP_RGBA* et passer au pixel suivant.

^avoir ce qui est vérifié par les assertions pour ce types de blocs

Vous aurez donc à implémenter l'algorithme dans le corps de la méthode `QOIEncoder::encodeData`.

```
// QOIEncoder::encodeData's signature
public static byte[] encodeData(byte[][] image)
```

Cette fonction prend comme paramètre un tableau bidimensionnel dont le format est celui résultant de l'application de `ArrayUtils.image_to_channels` (décomposition en canaux et linéarisation, revoir au besoin la section 4.4.1).

Elle retourne un tableau de type `byte[]` qui représente la codification des pixels (du contenu de l'image, "data") en format **Quite Ok Image** (entête et signature non compris).

Attention:

- les différences entre pixels doivent être calculées en arithmétique modulaire ("wrap around", voir 8.4) ;
- pour des raisons d'efficacité des traitements, il est déconseillé d'utiliser la méthode `concat` à chaque fois que l'on encode un bloc. Il est préférable de consigner les blocs encodés dans un `ArrayList` dont les éléments ne seront concaténés qu'une seule fois, d'un coup, à la fin de l'encodage.

Les hypothèses présumées correctes (à vérifier par des assertions) pour `QOIEncoder::encodeData` sont :

- que `image` et `image[i]` ne valent pas `null` ;
- et que `image[i]` est de taille 4.

5.5 Création du contenu du fichier

Comme vous l'avez appris dans la section 5.1, un fichier **Quite Ok Image** est composé de trois parties (entête, contenu/données et signature). Il s'agit maintenant de finaliser l'encodeur de sorte à pouvoir créer la représentation complète d'une image au format **Quite Ok Image**. Il vous est demandé pour cela de compléter la méthode `QOIEncoder::qoiFile` qui prend comme paramètre une image java (de type `Helper.Image`) et retourne un tableau de type `byte[]`) au format **Quite Ok Image** :

```
// QOIEncoder::qoiFile's signature
public static byte[] qoiFile(Helper.Image image)
```


Attention, cette méthode ne doit pas créer le fichier dans le disque, c'est le rôle de `Helper::write` (voir des exemples d'utilisation dans le programme `Main.java` fourni). Vous noterez que la signature est fournie par le biais de la constante `QOI_EOF`. L'hypothèse présumée vérifiée pour `QOIEncoder::qoiFile` est que `image` ne vaut pas `null`.

5.6 Tests

Pour tester le comportement de votre encodeur, utilisez le programme `Main.java`. Les exemples qui y sont fournis ne sont pas exhaustifs. **Complétez-les** selon ce qu'il vous semble judicieux pour tester l'ensemble des méthodes implémentées dans cette deuxième partie du projet.

Notez que le matériel fourni vous permet aussi de scruter vos données et de les comparer à des données de référence fournies dans le répertoire `references`.

Si vous vous êtes équipés pour visualiser les fichiers `.qoi` (voir la section 3.3), les images dans vos fichiers `.qoi` devraient être identiques à ceux des `.png` correspondants.

La comparaison visuelle d'image ne suffit pas à débusquer les erreurs facilement. Vous pouvez alors avoir recours à d'autres utilitaires.

Par exemple, pour vérifier le code de la méthode calculant l'entête, vous pouvez écrire dans `Main`:

```
Hexdump.hexdump(QOIEncoder.qoiHeader(Helper.read_image("references/beach.png")));
```

Ce qui devrait afficher dans la console le "dump":

```
=====
000000 : 71 6F 69 66 00 00 06 40 00 00 | qoif...@.. |
00000A : 04 B0 04 00 | ... |
=====
```

Il s'agit de la valeur (en base hexadécimale) des `bytes` tel que calculés par votre méthode `qoiHeader` sur l'image `references/beach.png`. Si vous ouvrez cette dernière, vous verrez qu'elle est de taille 1600x1200. Les 4 premiers `bytes` 71 6F 69 66 correspondent à l'encodage unicode des lettres q,o,i et f, les 4 `bytes` suivants 00 00 06 40 correspondent à la valeur 1600, les 4 `bytes` suivants à la valeur 1200, l'avant dernier `byte` vaut 4 (4 canaux) et le dernier `byte` vaut 0 (code de l'espace de couleur). (Utilisez des convertisseurs tel que [celui-ci](#) ou [celui-là](#) pour vous en convaincre).

Une petite astuce utile au débogage consiste à utiliser la facilité offerte par IntelliJ de faire afficher les valeurs en format hexadécimal ou binaire: dans le débogueur, faites un clic droit sur la une valeur de type entier puis `View as > Hex` ou `View as > binary`.

Pour vous faciliter la tâche, nous vous fournissons aussi des petites images très basiques (`qoi_op_*.png/.qoi`) qui vont être encodées plutôt par tel ou tel types de blocs (le noms de fichiers indique de quels types de blocs il s'agit). Vous disposez aussi d'une petite image `qoi_encode_test.png` dont l'encodage se fait en utilisant une fois chacun des 6 blocs possibles (`QOI_OP_RGBA`, `QOI_OP_RGB`, `QOI_OP_DIFF`, `QOI_OP_LUMA`, `QOI_OP_INDEX` et `QOI_OP_RUN`).

Le dossier `references` comporte les fichiers `.qoi` que vous êtes censés obtenir pour ces fichiers ainsi que pour ceux de plus grandes images. Ce dossier contient aussi les "dump" de référence censé être obtenus pour les plus petits fichiers (donnés dans des fichiers `.._dump.txt`). Vous pouvez les comparer visuellement pour détecter des anomalies. Par exemple, vous pouvez exécuter:

```
Hexdump.hexdump(QOIEncoder.qoiFile(Helper.read_image("references/qoi_op_run.png")));
```

et comparer visuellement ce que vous obtenez avec le contenu du fichier `references/qoi_op_run_dump.txt`. À noter que l'information dans le "dump" de référence, débarrassée des parties entête et signature, peut vous aider à déboguer les méthodes intermédiaires d'encodage.

Vous pouvez d'ailleurs "purger" la signature et l'entête d'une donnée encodée en procédant comme suit :

```
var encoding = QOIEncoder.qoiFile(image);  
Hexdump.hexdump(encoding, QOISpecification.HEADER_SIZE, encoding.length -  
    QOISpecification.QOI_EOF.length);
```

Vous pouvez enfin, en utilisant l'utilitaire `diff`, comparer votre résultat (produit dans le dossier `res/`) avec celui attendu fourni dans le dossier `references/`). Par exemple:

```
Diff.diff("references/qoi_op_rgba.qoi", "res/qoi_op_rgba.qoi");
```

6 Tâche 3 : Décodeur “Quite Ok Image”

Comme pour chaque protocole ou format, il ne suffit pas de savoir encoder. Le processus inverse et complémentaire de décodage est tout aussi important.

Vous avez vu à l'étape précédente différentes façons d'encoder les pixels d'une image. Pour chacune d'elles, à une exception près, vous aurez maintenant à programmer sa contrepartie "décodage". En clair, vous aurez à écrire une fonction pour décoder un bloc d'entête, une fonction pour décoder un bloc de type `QOI_OP_RGB`, etc.

6.1 Décodage de l'entête

L'entête d'un fichier **Quite Ok Image** contient des données qui sont cruciales pour le décodage. Pour pouvoir les extraire, il vous est demandé d'implémenter la méthode `QOIDecoder::decodeHeader`.

```
// QOIDecoder::decodeHeader's signature
public static int[] decodeHeader(byte[] header)
// Example
byte[] header = {'q', 'o', 'i', 'f', 0, 0, 0, 64, 0, 0, 0, 32, 3, 0};
// decoded = [64, 32, 3, 0]
int[] decoded = QOIDecoder.decodeHeader(header);
```

Pensez à utiliser ce que vous avez codé dans `ArrayUtils`. `HEADER_SIZE` est fournie dans `QOISpecification.java` et correspond à la taille attendue pour un bloc d'entête dans le format QOI.

Les hypothèses présumées correctes (à vérifier par des assertions) dans `QOIDecoder::decodeHeader` sont :

- que `header` ne vaut pas `null` ;
- que la taille du bloc `header` est conforme à la spécification (égal à la constante fournie `HEADER_SIZE`) ;
- que les 4 premiers `byte` sont égaux à la constante `QOI_MAGIC` ;
- que le nombre de canaux vaut `RGB` ou `RGBA` (constantes fournies) ;
- que l'espace de couleur vaut `ALL` ou `sRGB` (constantes fournies).

6.2 Algorithme de décompression QOI

L'algorithme de décompression a pour rôle de reformer les pixels à partir de leur représentation codifiée. Chaque type de bloc dans l'encodage QOI peut être décodé au moyen d'une fonction spécifique. C'est maintenant ces fonctions qu'il vous est demandé de programmer. Il s'agit des fonctions décrites ci-dessous. Le partie "**Tag**" de chaque bloc est ce qui va permettre de savoir quelle fonction invoquer dans l'algorithme global.

6.2.1 Décodage `QOI_OP_RGB`

Comme pour l'encodeur, le format `QOI_OP_RGB` est le plus facile à décoder. La fonction en charge de décoder ce type de bloc est à coder comme suit :

```
//QOIDecoder::decodeQoiOpRGB's signature
public static int decodeQoiOpRGB(byte[][] buffer, byte[] input, byte alpha,
    int position, int idx)
// Example
byte[][] buffer = new byte[2][4]; // buffer = [[0, 0, 0, 0], [0, 0, 0, 0]]
byte[] input    = {0, 0, 0, -2, 100, 0, 55, 8, 0, 0, 0};
byte alpha = 34;
int position = 0;
int idx = 4;
// returned_value = 3
int returned_value = QOIDecoder.decodeQoiOpRGB(buffer, input, alpha,
    position, idx);
// buffer = [[100, 0, 55, 34], [0, 0, 0, 0]]
```

Vous aurez remarqué que la signature de `QOIDecoder::decodeQoiOpRGB` est un petit peu plus complexe que la signature de `QOIEncoder::qoiOpRGB`. Les paramètres suggérés ont en fait pour but de faciliter l'intégration de cette fonction à l'algorithme global de décodage :

- `buffer` est l'image décodée en cours de construction ;
- `input` est la donnée (complète) à décoder ;
- `position` est la position où écrire les pixels décodés dans `buffer` ;
- `idx` est la position à laquelle lire les données dans `input`.

Comme la valeur de canal alpha est inconnue, elle doit être en paramètre (elle pourra ainsi être précisée lors de l'appel de la fonction). La valeur de retour est le nombre de `bytes` consommés dans `input`.

Les hypothèses présumées correctes (à vérifier par des assertions) pour `QOIDecoder::decodeQoiOpRGB` sont :

- que `buffer` et `input` ne valent pas la référence spéciale `null` ;
- que `position` pointe vers un emplacement valide de `buffer` ;
- que `idx` pointe vers un emplacement valide de `input` ;
- que `input` contient assez de données pour reformer le pixel.

6.2.2 Décodage *QOI_OP_RGBA*

De manière similaire à `QOIDecoder::decodeQoiOpRGB`, la méthode `QOIDecoder::decodeQoiOpRGBA` s'occupe de décoder le format *QOI_OP_RGBA*.

```
//QOIDecoder::decodeQoiOpRGBA's signature
public static int decodeQoiOpRGBA(byte[][] buffer, byte[] input, int
    position, int idx)
// Example
// buffer = [[0, 0, 0, 0], [0, 0, 0, 0]]
byte[][] buffer = new byte[2][4];
byte[] input     = {0, 0, 0, -2, 100, 0, 55, 8, 0, 0, 0};
int position = 0;
int idx = 3;
int returned_value = QOIDecoder.decodeQoiOpRGBA(buffer, input, position,
    idx);
// buffer = [[-2, 100, 0, 55], [0, 0, 0, 0]], returned_value = 4
```

Le rôle de cette méthode étant très similaire à celui de `QOIDecoder::decode_qoi_op_rgb`, nous nous passerons donc de le décrire.

Les hypothèses présumées correctes (à vérifier par des assertions) pour `QOIDecoder::decodeQoiOpRGBA` sont :

- que `buffer` et `input` ne valent pas la référence spéciale `null` ;
- que `position` pointe vers un emplacement valide de `buffer` ;
- que `idx` pointe vers un emplacement valide de `input` ;
- que `input` contient assez de données pour reformer le pixel.

6.2.3 Décodage *QOI_OP_INDEX*

Aucune méthode ne sera consacrée au décodage de *QOI_OP_INDEX*, il vous reviendra de la gérer proprement lors de l'implémentation de l'algorithme global.

6.2.4 Décodage *QOI_OP_DIFF*

Comme décrit dans la section 5.3.4, il nous suffit de connaître le pixel précédent ainsi que les variations dans chaque canal pour reconstituer le pixel suivant. La méthode chargée de ce traitement et qu'il vous faut coder est :

```
// QOIDecoder::decodeQoiOpDiff's signature
public static byte[] decodeQoiOpDiff(byte[] previous_pixel, byte chunk)
// Example
byte[] previous_pixel = {23, 117, -4, 7};
byte chunk            = (byte) 0b01_11_11_11;
var current_pixel = QOIDecoder.decodeQoiOpDiff(previous_pixel, chunk);
// current_pixel = [24, 118, -3, 7]
```

Les hypothèses présumées correctes (à vérifier par des assertions) pour `QOIDecoder::decodeQoiOpDiff` sont :

- que `previous_pixel` diffère de la référence spéciale `null` ;
- que la taille de `previous_pixel` soit égale à 4 ;
- que le "tag" de `chunk` a bien pour valeur `QOI_OP_DIFF_TAG` .

6.2.5 Décodage *QOI_OP_LUMA*

De manière similaire à la partie précédente, la méthode `QOIDecoder::decodeQoiOpLuma` aura pour rôle de décoder la partie *QOI_OP_LUMA*.

```
public static byte[] decodeQoiOpLuma(byte[] previous_pixel, byte[] data)
// Example
byte[] previous_pixel = {23, 117, -4, 7};
byte[] chunk          = {(byte) 0b10_10_01_01, (byte) 0b11_00_11_01};
byte[] current_pixel  = QOIDecoder.decodeQoiOpLuma(previous_pixel, chunk);
// current_pixel = [32, 122, 6, 7]
```

Elle fonctionne de manière similaire à `QOIDecoder::decodeQoiOpDiff`.

Les hypothèses présumées correctes (à vérifier par des assertions) pour `QOIDecoder::decodeQoiOpLuma` sont :

- que `previous_pixel` et `data` diffèrent de la référence spéciale `null` ;
- que la taille de `previous_pixel` soit égale à 4 ;
- que le "tag" de `data` vaut bien `QOI_OP_LUMA_TAG`.

6.2.6 Décodage *QOI_OP_RUN*

Pour finir, la méthode `QOIDecoder::decodeQoiOpRun` décode l'encodage de type *QOI_OP_RUN*. Elle prend comme paramètre le buffer à remplir, le pixel à reproduire, l'encodage à décoder et enfin, la position à partir de laquelle commencer à écrire dans le buffer. La valeur retournée correspond au nombre de pixels reproduits dans le buffer - 1 .

```
public static int decodeQoiOpRun(byte[][] buffer, byte[] pixel, byte chunk,
    int position)
// Example
byte[][] buffer = new byte[6][4]; // Array is full of zeros
byte[] pixel    = {1, 2, 3, 4};
byte chunk      = -61;
int position    = 1;
int returned_value = QOIDecoder.decodeQoiOpRun(buffer, pixel, chunk,
    position);
// buffer = [[0, 0, 0, 0], [1, 2, 3, 4], [1, 2, 3, 4], [1, 2, 3, 4], [1, 2,
    3, 4], [0, 0, 0, 0]]
// returned_value = 3
```

Les hypothèses présumées correctes (à vérifier par des assertions) pour `QOIDecoder::decodeQoiOpRun` sont :

- que `buffer` ne vaut pas la référence spéciale `null` ;
- que `position` pointe vers un emplacement valide de `buffer` ;
- que `pixel` ne vaut pas la référence spéciale `null` ;
- que `pixel` est constitué de 4 canaux ;
- que `buffer` contient assez d'espace pour reproduire le pixel.

6.3 Décodage global des données

Les méthodes de décodage des différent types de blocs doivent maintenant concourir à construire la représentation décodée des données. De manière tout à fait analogue à `QOIEncoder::encode_data`, il vous est maintenant demandé d'implémenter la méthode :

```
// QOIDecoder::decode_data's signature
public static byte[] [] decode_data(byte[] data, int width, int height)
```

Cette dernière prendra comme paramètre la partie “data” d’un fichier “Quite Ok Image” et procédera au décodage pour former les pixels. Attention, le format du tableau retourné ne correspond pas encore à celui d’une image de taille `width` x `height`. Le résultat est encore linéarisé et décomposé selon les 4 canaux (R,G,B et A), c’est à dire que le tableau retourné aura comme taille : `tab[width * height][4]`. Chaque `tab[pos]` est un pixel (tableau de 4 bytes).

Algorithme de décompression ”Quite Ok Image”:

Cet algorithme a pour rôle de décoder la partie “data” d’un fichier QOI. Il permet de reconstruire l’image au même format que l’entrée de (`QOIEncoder::encode_data`).

Étape 1 (Initialisation) :

De manière similaire à l’algorithme de compression, vous aurez à définir un ensemble de variables formant l’environnement d’exécution de l’algorithme. Ces variables sont similaires à celles définies dans l’algorithme de compression (le pixel précédent, la table de hachage etc.)

L’environnement de départ devra être initialisé à l’aide des valeurs prédéfinies fournies dans `QOISpecification` (`START_PIXEL`, `QOI_OP_RGB_TAG`, ...). D’autres variables peuvent venir se greffer à votre environnement si vous le souhaitez.

La table de hachage sert ici à stocker la valeur des pixels décodés.

Étape 2 (Traitement des données) :

Les données à décoder sont parcourues selon un indice `idx` qui évolue pour se placer à chaque fois au début d’un nouveau bloc. Dans cette configuration, le `byte` à la position `idx` en entier peut être un **Tag** (celui des blocs de type `QOI_OP_RGB` ou `QOI_OP_RGBA`). Il faut alors appeler les méthodes de décodage correspondantes et les utiliser de façon adéquate pour que `idx` soit positionné au début du prochain bloc.

Si l’intégralité du `byte` n’est pas un **Tag**, ses 2 premiers bits sont à observer pour déterminer de quel **Tag** il s’agit. Encore une fois, en fonction de la valeur de ce dernier, il s’agit d’appeler la méthode de décodage adaptée en mettant à chaque fois à jour l’indice de progression dans les données à décoder.

Au fur et à mesure que l’on progresse, on stocke dans la table de hachage les pixels décodés à la position qui leur est dévolue selon la fonction de hachage. C’est ce qui permettra de retrouver la valeur du pixel des blocs de type `QOI_OP_INDEX`.

Toutes les fonctions de décodage ne sont pas définies, et il vous revient de définir le code manquant pour compléter votre algorithme. Après l’invocation de toute méthode de décodage (qui résulte dans le décodage d’un pixel), vous aurez à mettre à jour les variables utiles (la table de hachage, le pixel précédent, etc).

Les hypothèses à vérifier pour `QOIDecoder::decode_data` sont :

- que `data` diffère de la référence spéciale `null` ;
- que `width` et `height` sont des valeurs positives ;
- que `data` contient assez de données pour reformer l'image de base.

6.4 Création de l'Image java

Maintenant que toutes nos méthodes ont été implémentées. Il nous reste à les regrouper pour pouvoir utiliser notre décodeur. Pour cela, vous aurez à implémenter la méthode `QOIDecoder::decode_qoi_file`. Elle prend comme paramètre le contenu du fichier et retourne une nouvelle image.

```
// QOIDecoder::decode_qoi_file's signature
public static Image decode_qoi_file(byte[] content)
```

Pour rappel, la méthode `Helper::generate_image` a pour rôle de créer une nouvelle image du type `Image`

Les hypothèses à vérifier pour `QOIDecoder::decode_qoi_file` sont :

- que `content` diffère de `null` ;
- que la signature de fin de fichier (`QOISpecification.QOI_EOF`) n'est pas corrompue.

6.5 Tests

Pour tester le comportement de votre décodeur, utilisez le programme `Main.java`. Les exemples qui y sont fournis ne sont pas exhaustifs. **complétez-les** selon ce qu'il vous semble judicieux pour tester l'ensemble des méthodes implémentées dans cette dernière partie du projet.

7 Extensions

Il est permis de compléter le code avec des extensions libres. Si vous avez une idée d'extension, merci de nous la soumettre via un post privé sur le [forum Ed](#) du cours, pour validation. Les extensions peuvent compenser des points perdus sur les parties obligatoires, mais la note globale du projet reste plafonnée à 6.

8 Complément théorique

8.1 Représentation ARGB

La représentation **ARGB** d'un pixel à l'aide d'un entier se fait de la manière suivante :

La couleur est décomposée en 4 composants ayant une valeur comprise entre 0 et 255 :

- **Alpha** est l'opacité du pixel. Si elle est à 0 alors le pixel est « invisible », peu importe ses autres composants. Si elle est entre 1 et 254 alors il est transparent et laisse passer les couleurs de l'image derrière celui-ci. Si sa valeur est 255, alors le pixel est parfaitement opaque.
- **Red** est la valeur d'intensité de la lumière rouge du pixel.
- **Green** est la valeur d'intensité de la lumière verte du pixel.
- **Blue** est la valeur d'intensité de la lumière bleue du pixel.

En considérant que le bit de poids le plus faible d'un entier ait pour indice 0 et que le bit de poids fort ait pour indice 31, alors ces 4 composants sont placés sur un même entier de telle sorte : les bits 31 à 24 définissent la valeur alpha, les bits 23 à 16 la valeur rouge, les bits 15 à 8 la valeur verte et les bits 7 à 0 la valeur bleue. Par exemple, si l'on veut représenter un pixel rouge en ARGB on a besoin que le composant alpha et rouge soit au maximum 255. Ce qui nous donne :

Alpha	Rouge	Vert	Bleu
255	255	0	0
0xFF	0xFF	0x00	0x00
11111111	11111111	00000000	00000000
31 24	23 16	15 8	7 0

La manière la plus pratique pour représenter une couleur en Java est d'utiliser l'écriture hexadécimale (plus concise). Par exemple pour définir le pixel rouge nous pouvons écrire :

```
int red = 0xFF_FF_00_00;
```

Les opérateurs «bitwise» permettent d'extraire facilement des valeurs de canaux à partir de la représentation sous la forme d'un entier (voir exemple sur la page suivante).

```
// Notre couleur en binaire
int x = 0b00000000_00100000_11000000_11111111;
// -> 2146559 (0x20c0ff)

// Décale de 8 bits vers la droite
int y = x >> 8; // -> 0b00000000_00100000_11000000

// ET binaire, ce qui a pour effet de ne garder que les 8
// premiers bits
int z = y & 0b11111111; // -> 0b11000000
// On a bien récupéré notre composante verte à 192
// on aurait aussi pu écrire: int z = y & 0xff;
```

8.2 Table de hachage

Une table de hachage est une structure de données qui permet de créer une structure associative de type clé-valeur : la table contient un ensemble de paires clé-valeur. La notion de clé généralise la notion d'indice telle que vous la connaissez dans un tableau ordinaire. Une clé peut en effet être un objet quelconque.

Pour pouvoir accéder efficacement à l'entrée correspondant à une clé donnée dans le tableau, on a recours à une fonction dite de *hachage*.

Hachage : Le hachage est la transformation d'un objet quelconque (clé) en un entier. Cette opération est à la base du fonctionnement des tables de hachage.

Problème de collisions: Un des problèmes du hachage est la *collision*. Ce phénomène se produit lorsque deux objets différents, produisent le même entier lors du hachage. Plusieurs solutions existent lors l'implémentation d'une table de hachage pour résoudre ce problème et libre au programmeur de choisir laquelle utiliser en fonction de ses besoins.

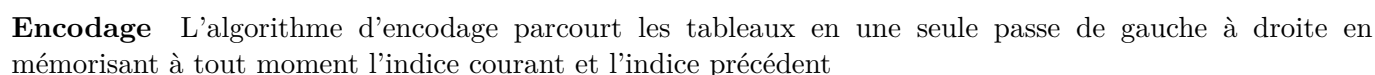
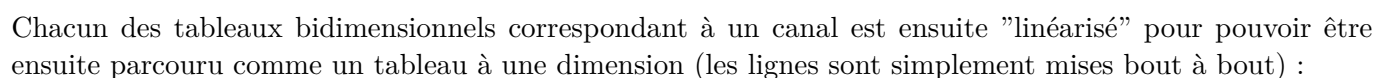
Dans le projet Java offre des structures de données prédéfinies pour mettre en oeuvre le concept de table de hachage. Ceci dépasse le cadre de ce cours et vous aurez l'occasion de découvrir ce matériel en détail au semestre prochain. Pour ce projet, nous retenons le principe du hachage, mais appliqué à un tableau classique : l'indice auquel stocker une donnée est calculé par une fonction de hachage appliquée à cette donnée. La fonction de hachage est quant à elle fournie.

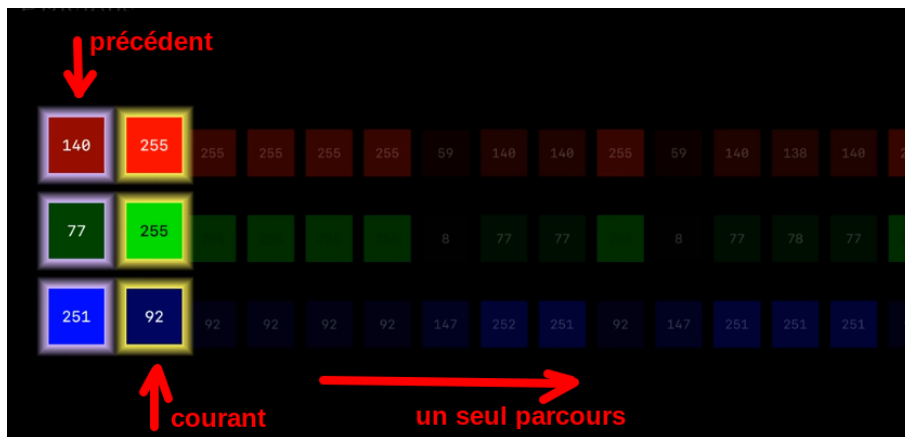
8.3 Le format "Quite Ok Image"

Une image peut être représentée comme un tableau à deux dimensions de pixels, c'est à dire un tableau de valeurs entières au format RGBA. Lorsque l'image est de taille importante, stocker explicitement chaque pixel peut rapidement devenir coûteux en terme d'espace.

Le format "Quite Ok Image" vise à codifier l'information contenue dans une image de façon plus économique. Il se base sur l'idée que très souvent dans une image, des pixels voisins sont identiques ou très proches l'un de l'autre en terme de couleur.

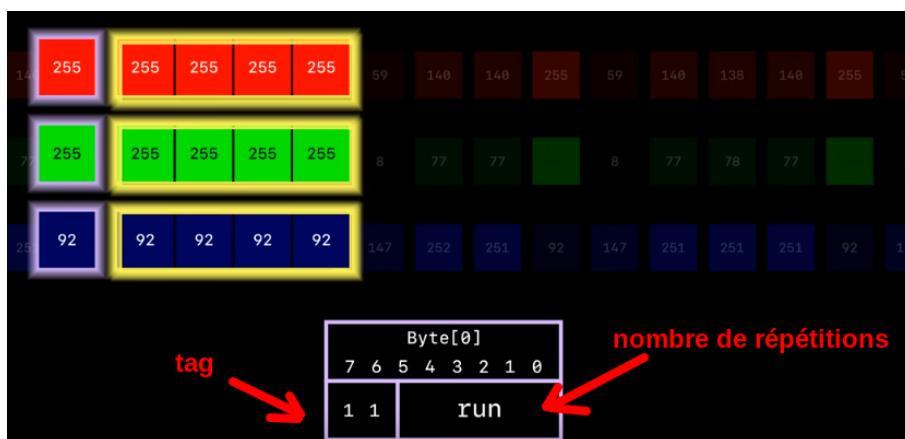
Décomposition de l'image Pour des motifs d'optimisation et de simplification du parcours des données, chaque image est d'abord décomposée, pixel par pixel, selon les canaux R,G,B et A (ici pour simplifier seule la décomposition selon R, G et B est montrée):





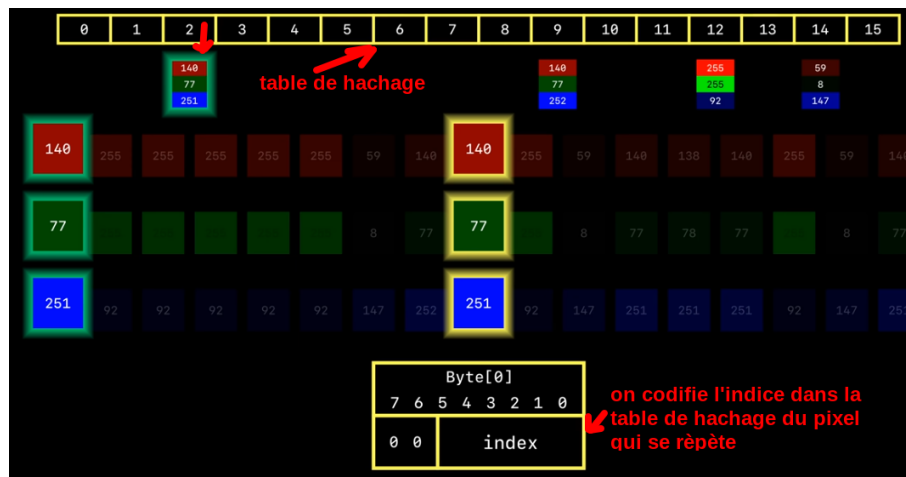
En fonction de la valeur des pixels à ces indices il est décidé d'encoder l'information au moyen de divers types de blocs. Il existe 6 types différents de blocs, décrit dans la [spécification du format QOI](#).

Par exemple le bloc `QOI_OP_RUN` sert à coder le nombre de répétitions d'un pixel identique :



Ainsi au lieu de stocker n fois le même pixel on va uniquement stocker le nombre n (plus le minimum d'informations utiles pour pouvoir par la suite décoder proprement). Chaque type de bloc est en effet identifié par un *tag* qui lui est spécifique (ce qui va permettre le décodage).

En cours d'encodage, les 64 derniers pixels rencontrés sont par ailleurs stockés dans une *table de hachage*. Si un pixel stocké dans la table vient à apparaître une autre fois dans l'image, on n'encodera pas ce pixel explicitement, mais par biais d'un bloc `QOI_OP_INDEX` qui stocke l'indice du pixel dans la table de hachage; ce qui est beaucoup plus concis :



L'indice du pixel dans la table de hachage est donné par une fonction de hachage particulière et seuls les pixels qui ne causent pas de collision sont encodés au moyen d'un bloc de type "QOI_OP_INDEX" (les autres seront encodés au moyen d'autres types de blocs).

Les images utilisées ci-dessus sont tirées de cette vidéo: <https://www.youtube.com/watch?v=EFUYNoFRHQI>. Vous y trouverez depuis la minute 23' environ, une présentation générale complète du principe de fonctionnement de l'encodage en format "Quite Ok Image" (en anglais).

Les explications utiles pour programmer l'encodage/décodage en format QOI sont données dans l'énoncé ainsi que dans les transparents du cours.

8.4 Entiers signés et entiers non signés

En mémoire, un entier n'est qu'une simple séquence de bits. Sa valeur effective dépend entièrement de la façon dont les bits qui le composent sont interprétés. Il existe plusieurs façons d'interpréter une séquence de bits, chacune d'elles ayant ces propres bénéfices et les plus connus étant : «2's complement» (Le complément à deux), «1's complement», «sign and magnitude» et «offset binary».

En java, les entiers sont tous encodés sous forme de *complément à deux*. C'est à dire que les entiers en java peuvent être négatifs et qu'en ajoutant 1 à la plus grande valeur représentable on obtient 0 et qu'en retranchant 1 à la plus petite valeur possible on obtient la plus grande valeur possible. Contrairement à d'autres langages de programmation, comme le C, il n'est pas possible de désactiver cette interprétation ce qui peut impliquer certaines difficultés.

Pour ce projet, ceci se traduit en une limitation intrinsèque de l'outil de compression que vous allez développer. En effet, si l'image est de très grande taille (type `int` insuffisant), l'entête du fichier "Quite Ok Image" pourra comporter un nombre négatif pour l'information sur la taille. Cette limitation est délibérément acceptée par simplification.

Voici un résumé des points importants à retenir lors du codage du projet pour ce qui est de la représentation des entiers :

- les canaux du format ARGB prennent comme valeurs des entiers non signés entre 0 et 255 (ce qui nécessite 1 octet) ;
- le type entier `byte` est codé sur un octet, mais ses valeurs sont toujours considérées comme signées en Java et elles sont représentées en complément à deux ;
- en complément à deux l'addition et la soustraction sur les nombres binaires se font en arithmétique modulaire («wrap around»), c'est à dire qu'elles se font modulo le nombre maximal représentable et

qu'il n'y a pas de débordement. Certains encodages que vous implémenter nécessitent de calculer des soustractions entre valeurs de canaux (nombres valant entre 0 et 255) et l'ajout de *décalages* à ces valeurs : **il est attendu que ces additions/soustractions se fassent en arithmétique modulaire.**

Une astuce pour permettre d'obtenir les valeurs en arithmétique modulaire des additions et soustraction entre nombres binaires compris entre 0 et 255 et de les convertir en `byte`:

```
byte b = (byte)255;
System.out.println(b); // - (2^8 - 255) = -1
b = (byte)248;
System.out.println(b); // - (2^8 -248) = -8

// Arithmétique modulaire ("wrap around")
b = (byte)(255 + 1);
System.out.println(b); // 0
b = (byte)-255;
System.out.println(b); // +1
byte b1 = 127;
byte b2 = -128;
b = (byte)(b2-b1);
System.out.println(b); // +1
```

Pour la représentation en complément à 2, vous pouvez regarder la vidéo «Représentation binaire des nombres entiers» par Olivier Lévêque (<https://tube.switch.ch/videos/JWKOU3kEA1>) ou la vidéo «Représentation des entiers en binaire» de Ronan Boulic (<https://youtu.be/a5gLSc0tbjI>).

En java pour convertir un entier en sa valeur non signée vous pouvez avoir recours à différents procédés simples (voir par exemple [ceci](#)).